

Project Link: https://github.com/mkaramica/estimate_carPrice

1- Project Overview

The objective of this project was to create a simple internal search interface that estimates the average market value for a year/make/model of a used car. The interface would also allow for optional search criteria such as mileage and the number of returned records.

The project utilized a database with 4,713,914 records of used cars listed in the market, and the front-end was created using *HTML*, *CSS*, *JavaScript*, and *Bootstrap*. The back-end part was built using *FLASK* library in *Python*, with *MySQL* used as the database management system. The server used was *CentOS 7*. The linear regression model was implemented for price estimation.

2- How to Utilize the Web Application

The web application is run on *CentOS 7* server, which is a *Linux* operating system. *Gunicorn*, a well-known *Python* WSGI (Web server Gateway Interface), is utilized to run the web application. The application is located in `~/mkarami/estimate_carPrice/` path on the server. The *Python* dependencies are installed in a virtual environment located with the same folder named as *py_env*, which should be activated before running the program via the following Linux command.

```
source py_env/bin/activate
```

The web application can then be executed using *gunicorn* with the following command, which launches the web application on port 8000 of the local host, i.e., *localhost:8000*.

```
gunicorn app:app
```

One can alternatively run a bash script located on the root of the provided server to launch the web application, as follows.

```
./run_script.sh
```

There is no graphical user-interface installed on the provided server. Thus, an external link should be provided to visit the running website from the outside of the server. The following command will enable the user to access the web application on port 8000 of the client's system. This command should be executed on the client's computer (not on the server).

```
ssh -L 8000:localhost:8000 root@104.238.228.196  
[Password:] r*ga5&jTH4E8
```

After running the mentioned command, the web application will launch on <http://localhost:8000/> address of the client's computer.

3- User Interface

The user interface provides the users with the capability to search for the price of desired vehicle with options to either enter all the information in one line (as requested in the project requirements), or input the year, make and model as separated fields. For simple make/model combinations, there is no difference between the options as they can be retrieved easily from the one-line input by splitting the text with the spaces. However, regarding the actual cases existing in the database, there might be multiple cases for the makes with a space in their name (e.g., *Alfa Romeo*), or in their model (e.g., *Toyota Land Cruiser*). Moreover, the given database requires major cleansing as there are several wrongly entered items, that makes it even more difficult to retrieve the makes and models from one-line input. Thus, two described options were considered for the user for entering the year/make/model items accordingly. The mileage field is optional for both cases, according to the project requirements.

[Clear URL](#)

Please enter the required items!

Vehicle Information

Enter Information as ...

☒ One Combined Line
☐ Separated Fields

Year Make Model* [Required]

Mileage [Optional]

Maximum Number of Records [Optional]

Submit

Developed by Mahdi Karami
mahdi.karami.ca@gmail.com

[Clear URL](#)

Please enter the required items!

Vehicle Information

Enter Information as ...

☐ One Combined Line
☒ Separated Fields

Year* [Required]

Make* [Required]

Model* [Required]

Mileage [Optional]

Maximum Number of Records [Optional]

Submit

Developed by Mahdi Karami
mahdi.karami.ca@gmail.com

By entering the required fields and clicking the submit button, the user will be guided to the results page if everything is OK and there is at least one record for the requested query.

A snapshot of a results page is presented in the figure below, where the estimated price of an “2020 Honda Accord with a mileage of 33,000 Miles” is provided by considering 100 similar cases. The rest of the records (item 14 to 100) are cropped from the image.

localhost:8000/?info_input_type=one-line&year_make_model=2020+Honda+Accord&year=&make=&model=&mileage=33000&max_records=100

Go Back to the Search Page

Your Requested Vehicle

Year	Make	Model	Mileage
2020	Honda	Accord	33000 Miles

Estimated Price \$31000

Queried Cases

#	Vehicle	Price	Mileage	Location
1	2020 Honda Accord SPORT SEDAN 4D	\$30975	33002 Miles	Alhambra, CA
2	2020 Honda Accord Sport	\$31989	33002 Miles	Thornton, CO
3	2020 Honda Accord EX-L	\$34000	32961 Miles	Lisle, IL
4	2020 Honda Accord Touring 2.0T	\$35237	32956 Miles	Jacksonville, FL
5	2020 Honda Accord LX	\$30890	32919 Miles	Folkston, GA
6	2020 Honda Accord 1.5T LX FWD	\$27998	33084 Miles	Danvers, MA
7	2020 Honda Accord 2.0T Touring FWD	\$34998	32913 Miles	Puyallup, WA
8	2020 Honda Accord Sport	\$31000	33091 Miles	Atmore, AL
9	2020 Honda Accord LX	\$25999	33122 Miles	Santa Monica, CA
10	2020 Honda Accord EX	\$29732	32839 Miles	Rochester, NY
11	2020 Honda Accord Sport	\$31571	32831 Miles	Rockville, MD
12	2020 Honda Accord 1.5T LX FWD	\$27998	32794 Miles	Houston, TX
13	2020 Honda Accord Sport	\$32995	33224 Miles	Tucson, AZ

The following figure demonstrates the snapshot of the results of looking for the price of an “2016 Toyota Camry” without mentioning the mileage. The rest of the records (item 14 to 100) are cropped from the image.

localhost:8000/?info_input_type=separated-fields&year_make_model=&year=2016&make=Toyota&model=Camry&mileage=&max_records=100

Go Back to the Search Page

Your Requested Vehicle

Year	Make	Model	Mileage
2016	Toyota	Camry	Not Given

Estimated Price \$18800

Queried Cases

#	Vehicle	Price	Mileage	Location
1	2016 Toyota Camry Special Edition	\$18795	50770 Miles	Holiday, FL
2	2016 Toyota Camry SE	\$18786	79486 Miles	Summerville, SC
3	2016 Toyota Camry LE	\$18818	61000 Miles	Sacramento, CA
4	2016 Toyota Camry Special Edition	\$18777	122154 Miles	South Gate, CA
5	2016 Toyota Camry SE	\$18765	87570 Miles	Columbus, OH
6	2016 Toyota Camry SE	\$18750	70410 Miles	Kansas City, KS
7	2016 Toyota Camry LE	\$18870	70923 Miles	San Jose, CA
8	2016 Toyota Camry LE	\$18729	55153 Miles	Danvers, MA
9	2016 Toyota Camry SE	\$18715	98408 Miles	San Jose, CA
10	2016 Toyota Camry Special Edition	\$18888	91561 Miles	Show Low, AZ
11	2016 Toyota Camry Special Edition	\$18710	76067 Miles	Cockeysville, MD
12	2016 Toyota Camry Special Edition	\$18900	118859 Miles	Sonora, CA
13	2016 Toyota Camry LE	\$18900	51087 Miles	Lewisville, TX

4- Database Overview

The database used for this project contains information about the market value of used cars. The raw data was sourced from the following document (provided by *VinAudit*) and consisted of 4,713,914 records. The data was in a raw text format and needed to be processed and converted into a database format.

Full Market Data File (2021-07-07)	https://drive.google.com/file/d/149kLDX0j0NTnHBannuoMTBjisENmGq6o/view
---------------------------------------	---

In the raw text-formatted file provided by *VinAudit*, there are 25 different features for each record, separated by the vertical bar character, (“|”). To reduce the size and execution time of the queries, only 9 columns were inserted into *MySQL* database. There is a *Python* script that converts the raw text-formatted data into *MySQL* database, existing in the following path.

~/mkarami/estimate_carPrice/utils/dataIntoMySQL.py

The authentication information of the *MySQL* database used for the web application is given in the following table.

Item	Value
MySQL host	localhost
MySQL username	root
MySQL Password	Admin13852/2
MySQL database Name	vinaudit_database

Important Note: The authentication information is stored in the application pass on the server, which is available at the following address. The file is hidden and can only be seen with “*ls -a*” command in the Linux terminal.

~/mkarami/estimate_carPrice/.env

One can simply use “*nano .env*” command in the terminal to see or modify the content of this file.

- Database Schema

There is a single table in the database, called “*cars*”, which includes 10 columns as follows.

FIELD	TYPE	NULL	KEY	INDEX
vin	varchar(30)	No	PRI	PRIMARY
year	int	Yes	MUL	idx_year
make	varchar(200)	Yes	MUL	idx_make
model	varchar(200)	Yes	MUL	idx_model
trim	varchar(200)	Yes		
price	int	Yes		
mileage	int	Yes		
city	varchar(200)	Yes		
state	varchar(20)	Yes		
make_model	varchar(300)	Yes	MUL	idx_make_model

The primary key for the table is the “vin” column, which is unique for each record in the table. There are four additional indexes defined on the frequently queried columns to reduce the execution time. It is worth mentioning that the defined indexes reduce the query execution times by a factor of up to 100 times, as a result of a relatively large database we deal with.

It should also be mentioned that in the table above, there are 9 independent columns, and the last column (“*make_model*”) is indeed a combination of “*make*” and “*model*” columns with a space in between. Although this might seem a table with redundant information which violates the database normalization rules, it enables us to accelerate the search query execution by defining an extra index on the combined *make_model* column, which provides search options described in the “User Interface” section.

- Data cleansing

There are several wrong data records in the database requiring comprehensive data cleansing process to increase the performance of the application for the future steps of the projects. The most important columns for data cleansing are make, model, and trim. For example, there are many cases where one of these fields is empty, and the data field is given in another column. For example, the make and model are written in the trim column. About 50,000 records have been identified for such issues and the corresponding records have been modified by *REGEX* technology. However, there are still many records in the table requiring data cleansing. The main issue with these wrong records is that they dramatically increase the number of existing make/model combinations, that reduces the performance of column indexing of the database.

5- Dataflow and Code Structure

The overall structure of the Python code is illustrated in the figure below, which contains three main classes, namely *PriceEstimation*, *DatabaseController*, and *WebApp*. The *index()* function is responsible for receiving the *HTTP* request and rendering the corresponding *HTML* content. The function *timer_decorator* is defined to give the runtime of the executed queries, which is useful to monitor the performance of the indexing and search algorithms.

```

1  # Author: Mahdi Karami; email: mahdi.karami.ca@gmail.com
2
3  from flask import Flask, render_template, request
4  import mysql.connector
5  from dotenv import load_dotenv
6  import time
7  import os
8
9
10 app = Flask(__name__)
11
12 @def timer_decorator(func):
28
29
30 @class PriceEstimation:
58
59
60 @class DatabaseController:
185
186
187 @class WebApp:
291
292
293 web_app = WebApp()
294
295 @app.route('/', methods=['GET'])
296 @def index():
318
319
320 if __name__ == '__main__':
321     web_app.run()
322

```

As soon as the user clicks the submit button on the web application, a “Get” *HTTP* request is sent to the server, that contains the features of the query of interest. Based on the options provided to the user, different items can exist in the “GET” request. The *index()* function receives the *HTTP* request and delivers it to an instance of *WebApp* class. There are multiple methods in *WebApp* that are responsible for evaluating the *HTTP* request and converting it to the required fields of the *SQL* queries. If the received *HTTP* request is valid, the corresponding query is performed; otherwise, a warning message will be sent back to the *index()* function to inform the user about the issue in the input fields.

The *WebApp* class includes a method to communicate with the *databaseController* class, called *perform_query*, that sends the requested query to the database, and receives the results to show them to the user. It somehow works as a channel between the database and the outer layer.

All the tasks related to the database is handled then by the *databaseController* class, including authentication, connection/disconnection, and query execution. There is also a separate class named *PriceEstimation*, that currently possesses a single function with static method decorator to implement one dimensional linear regression (i.e., least-square estimation) to calculate the price based on the given list of mileage-price pairs. Other advanced methods of price estimation, like multi-dimensional linear regression or other machine learning approaches, can be added to this class in the future.

The query results will be sent back to the *index()* function, where an *HTML* template is ready to be filled with the data. The user interface was developed by *HTML* technology while the styles are applied using *CSS*. To accelerate the design procedure, some available modern bootstrap styles were added to the designed pages as well. In addition, JavaScript was utilized to provide some functionalities in the front-end user interface.

6- Test Cases

Multiple test cases were defined to check the functionality of the user interface as well as the dataflow for different cases, whose report is available in the following google sheet.

<https://docs.google.com/spreadsheets/d/1ZeApDDylsJXpE5OEWoBdceXNAo2T5ymEuYSIz5wfiaY/edit#gid=0>

There are totally 14 test cases to check the functionality of the web application for searching with/without giving the mileage, using combined one-line input vs. separated fields, search for cases with a space in makes/models, and checking the upper/lower case sensitivity. All the tests were passed successfully.

7- Bonus: How to Improve the Estimation

7-1- Overall Discussion

Estimating the price of a used car can be a challenging task, as it depends on a variety of factors such as the make and model, year, mileage, and condition of the vehicle. While the project has provided a solid foundation for estimating the average market value of a used car, there are many methods to improve the accuracy of the estimation. These methods typically involve using more features and incorporating as much data as possible, in order to capture the complex relationships between different factors and the final price of the car. Some specific methods for improving accuracy include using a more advanced machine learning algorithm, incorporating external data sources such as regional market trends or sales history, and using a more complex data pre-processing pipeline that includes steps such as outlier detection and feature engineering. By continually evaluating and refining the model over time, it may be possible to improve the accuracy of the price estimation and provide a more accurate and valuable tool for buyers and sellers of used cars.

One potential defect of the implemented method for estimating the price of a used car is that the linear model is extracted from a very limited number of similar cars (for example, 100 records). While this approach may be effective for some cases, involving more records in the model could improve its accuracy by capturing a wider range of trends and patterns in the data. Additionally, the current method fixes the year in the estimation process, which may not accurately reflect the impact of market trends or pricing fluctuations in nearby years.

To address these limitations, a more comprehensive model could be developed that takes into account a wider range of factors, such as the condition of the vehicle, the number of previous owners, and the location and time of sale. This model could be designed to effectively combine all of the available features while avoiding excessive complexity, using advanced machine learning techniques such as decision trees, random forests, or neural networks. By leveraging a more comprehensive model, it may be possible to improve the accuracy and relevance of price estimations for used cars, and provide a more valuable tool for buyers and sellers alike.

I have carried out a similar project to perform multi-variable regression method for movie rating over a 10 million movie rating records, which is somehow comparable with the size of our database. The report is available in the following address on my *GitHub* account, addressing how we can extract the effect of different parameters to achieve a least squared estimation.

<https://github.com/mkaramica/movieRecommendationSystem/blob/main/MovieLensReport.pdf>

7-2- Proposed Generalized Linear Regression Approach

Here is a similar approach to go one step forward and consider the effects of year, mileage, state, and certified parameters for price estimation.

- Normalize all the prices by dividing them into the average price of the corresponding make model.

$$\tilde{P}_i = P_i / P_{avg}$$

- For each record, the normalized price is assumed to be the summation of the following terms.

$$\tilde{P}_i = \mu + B_{state} + B_{year} + B_{mileage} + B_{certified} + err_i$$

- μ is the average of normalized prices of all the records in the database.
- B_{state} is the biased term representing the overall difference in car prices in different states. For instance, the average price of cars in California is about 2,000\$ more than that of Michigan.
- B_{year} X shows how the price of all cars decreases over time.
- $B_{mileage}$ is an indicator to consider the effect of a car mileage on the price, achieved by all the records of the database.
- $B_{certified}$ represents the difference between the price of certified cars (as they tend to be more reliable) and the uncertified ones.
- err_i represents the estimation error indicating how much the actual normalized price of each record differs from the estimated price.
- The goal of any regression method is to minimize summation of squared errors of all the records in the database.

According to my experience in similar projects, the proposed algorithm can be implemented to improve the price estimation in 2-3 days, as per request.